

Module 9 — Spring Security

Highest priority for any real backend role. JWT auth flow, the filter chain, and authentication vs authorization are asked constantly. Be able to *draw* the auth flow.

Node.js bridge: Like Passport.js middleware + bcrypt + jsonwebtoken, but as a structured **filter chain** in front of your controllers, with a shared `SecurityContext` per request.

9.1 Authentication vs Authorization

- **Authentication (AuthN)** — *who are you?* Verify identity (username/password, token, OAuth).
- **Authorization (AuthZ)** — *what can you do?* Check permissions/roles after identity is known.
- Order: authenticate first, then authorize. In Spring, AuthN populates the `SecurityContext`; AuthZ checks it (URL rules or `@PreAuthorize`).

9.2 The Security Filter Chain (core internal question)

Core Concept

Spring Security is a **chain of servlet filters** inserted before the `DispatcherServlet` (via a single `DelegatingFilterProxy` → `FilterChainProxy`). Each filter has one job; the request flows through until authenticated + authorized, then reaches your controller.

Internal Working — key filters in order

```
Request
-> SecurityContextPersistenceFilter (load SecurityContext, e.g., from session)
-> [your JwtAuthenticationFilter] (parse token, set Authentication) <-- custom, stateless
-> UsernamePasswordAuthenticationFilter (form login: build auth token)
-> ... other filters (CORS, CSRF, logout) ...
-> ExceptionTranslationFilter (handle AuthN/AuthZ exceptions -> 401/403)
-> AuthorizationFilter (FilterSecurityInterceptor pre-6) (enforce access rules)
-> DispatcherServlet -> Controller
```

Authentication flow (form/password) — internal

```
1. UsernamePasswordAuthenticationFilter builds an (unauthenticated) Authentication token
2. -> AuthenticationManager (ProviderManager)
3. -> DaoAuthenticationProvider
   -> UserDetailsServiceImpl.loadUserByUsername() -> UserDetails (hash + authorities)
   -> PasswordEncoder.matches(raw, storedHash) (BCrypt)
4. success -> authenticated Authentication stored in SecurityContextHolder
   failure -> AuthenticationException -> 401
```

Memory Diagram

```
[SecurityContextHolder] (ThreadLocal per request)
  holds -> Authentication { principal(UserDetails), authorities[ROLE_*], authenticated=true }
Controller / @PreAuthorize reads it to authorize.
```

Key components (memorize)

Component	Role
<code>AuthenticationManager</code> / <code>ProviderManager</code>	orchestrates authentication
<code>AuthenticationProvider</code> (<code>DaoAuthenticationProvider</code>)	does the actual verification
<code>UserDetailsService</code>	loads user (hash + roles) by username
<code>UserDetails</code>	user data + authorities
<code>PasswordEncoder</code> (<code>BCryptPasswordEncoder</code>)	hash + verify passwords
<code>SecurityContextHolder</code>	ThreadLocal holding the current <code>Authentication</code>
<code>SecurityFilterChain</code>	the ordered filters (config bean in Spring Security 6)

9.3 JWT Authentication (the most-asked security topic)

Core Concept

JWT (JSON Web Token) = a signed, self-contained token: **Header.Payload.Signature** (base64url). Stateless — the server verifies the **signature** instead of a session lookup, so any instance can validate it (great for microservices).

Structure & flow

```
JWT = base64(header).base64(payload).base64(HMAC/RSA signature)
header : {"alg":"HS256","typ":"JWT"}
payload: {"sub":"user123","roles":["ADMIN"],"exp":...,"iat":...} (claims, NOT encrypted)
signature: HMACSHA256(header.payload, secret) <-- verifies integrity + authenticity

Login: POST /login {user,pass} -> verify -> return signed JWT (+ refresh token)
Call:  GET /api/... Authorization: Bearer <jwt>
      JwtFilter: verify signature + exp -> build Authentication -> SecurityContext
```

Internal Working — stateless request

1. Client sends Authorization: Bearer <token>
2. Custom `JwtAuthenticationFilter` extracts + validates token (signature, expiry, issuer)
3. Builds an authenticated `Authentication` (username + authorities from claims)
4. Sets it in `SecurityContextHolder` -> downstream authorization works
5. No server session -> horizontally scalable

Best Answer

“On login I verify credentials with BCrypt and issue a signed JWT whose payload carries the subject and roles. On each request a custom filter early in the chain validates the signature and expiry, rebuilds an `Authentication`, and puts it in the `SecurityContextHolder` — so auth is stateless and any service instance can verify it without a shared session store. The payload is signed, not encrypted, so I never put secrets in it, keep access tokens short-lived, and use refresh tokens for renewal.”

JWT traps (interviewers probe these)

- **Payload is not encrypted** — anyone can base64-decode claims. Never store secrets.
- **Can't revoke easily** — stateless means you need short expiry + a refresh/blacklist strategy for logout.
- Always validate `alg` (reject `none`), expiry, issuer/audience.
- Store carefully on the client (httpOnly cookie vs localStorage → XSS/CSRF trade-offs).

Coding Example (Spring Security 6 style)

```
@Configuration
@EnableWebSecurity
class SecurityConfig {
    @Bean
    SecurityFilterChain chain(HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless API -> CSRF off
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    AuthenticationManager authManager(AuthenticationConfiguration c) throws Exception {
        return c.getAuthenticationManager();
    }
}
```

9.4 BCrypt & PasswordEncoder

- **Never store plaintext or fast hashes (MD5/SHA-1)** for passwords. Use **BCrypt** (or Argon2/PBKDF2): a slow, salted, adaptive hash.
- BCrypt embeds a random **salt** and a **work factor** (strength, default 10) in the hash → same password yields different hashes; brute-force is deliberately slow.
- Verify with `encoder.matches(raw, storedHash)` — never decrypt (it's one-way).

9.5 CORS & CSRF (always confused — nail the difference)

	CORS	CSRF
Stands for	Cross-Origin Resource Sharing	Cross-Site Request Forgery
What	Browser policy: which origins may call your API	Attack: forged request using victim's cookies
Direction	Relaxes same-origin policy	Protects state-changing requests
Config	allowed origins/methods/headers	token per form/request
For JWT APIs	configure allowed origins	usually disabled (no cookies → no CSRF)

- **CORS** is not security — it's the browser deciding if JS from origin B may read responses from your API. Configure allowed origins/methods.
- **CSRF** matters for **cookie/session** auth (browser auto-sends cookies). With **stateless JWT in an Authorization header**, CSRF protection is typically disabled because there's no ambient cookie to forge.

Module 9 — Top 25 Interview Questions (senior answers)

1. **AuthN vs AuthZ?** Who you are vs what you can do.
2. **How does Spring Security work?** Ordered servlet filter chain before controllers.

3. **Key filter-chain components?** AuthenticationManager, provider, UserDetailsService, PasswordEncoder, SecurityContextHolder.
4. **Form-login auth flow?** Filter → ProviderManager → DaoProvider → UserDetailsService + encoder → SecurityContext.
5. **What is SecurityContextHolder?** ThreadLocal holding the current Authentication.
6. **What is UserDetailsService?** Loads user (hash + authorities) by username.
7. **What does AuthenticationProvider do?** Performs actual credential verification.
8. **JWT structure?** header.payload.signature (base64url).
9. **Is JWT encrypted?** No — signed; payload is readable. Don't store secrets.
10. **How is a JWT validated per request?** Custom filter verifies signature+expiry, sets Authentication.
11. **Why is JWT good for microservices?** Stateless; any instance verifies via signature.
12. **JWT revocation/logout problem?** Stateless → short expiry + refresh/blacklist.
13. **Access vs refresh token?** Short-lived API token vs long-lived renewal token.
14. **Why BCrypt over MD5/SHA?** Slow, salted, adaptive work factor.
15. **Does BCrypt use a salt?** Yes — embedded, random per hash.
16. **How to verify a password?** `encoder.matches(raw, hash)` (one-way).
17. **CORS vs CSRF?** Browser origin policy vs forged-request attack.
18. **Why disable CSRF for JWT APIs?** No cookies → nothing to forge.
19. **When keep CSRF on?** Cookie/session-based auth.
20. **Method vs URL security?** `@PreAuthorize/@Secured` vs `authorizeHttpRequests`.
21. **Roles vs authorities?** Role = `ROLE_`-prefixed authority; authorities are granular.
22. **How to make it stateless?** `SessionCreationPolicy.STATELESS`.
23. **401 vs 403?** Unauthenticated vs authenticated-but-forbidden.
24. **How does @PreAuthorize work?** AOP proxy evaluating SpEL against the Authentication.
25. **OAuth2/OIDC vs JWT?** Delegated auth protocol vs token format (often used together).

Module 9 — Top Coding Questions

- Implement a `JwtAuthenticationFilter` (extract, validate, set context).
- Configure a stateless `SecurityFilterChain` with role-based URL rules.
- Implement `UserDetailsService` backed by a JPA repository + BCrypt.
- Add `@PreAuthorize("hasRole('ADMIN')")` to a method and test 403.
- Build login endpoint issuing access + refresh tokens.

Module 9 — Common Follow-ups

- “How do you log out / revoke a JWT?” (short expiry, refresh rotation, denylist.)
- “Where do you store the token client-side and why?” (httpOnly cookie vs localStorage trade-offs.)
- “Someone base64-decodes your JWT and reads the roles — is that a vulnerability?” (No — it's signed, not secret; don't put secrets in it.)

Module 9 — One-Page Cheat Sheet

```
AuthN(who) then AuthZ(what). Filter chain before DispatcherServlet.
Flow: Filter -> AuthenticationManager(ProviderManager) -> DaoAuthenticationProvider
      -> UserDetailsService + PasswordEncoder(BCrypt) -> SecurityContextHolder(ThreadLocal)
JWT = header.payload.signature (signed, NOT encrypted). Verify signature+exp per request in a filter.
Stateless -> scalable; revoke via short expiry + refresh token. Never put secrets in payload.
BCrypt = slow, salted, adaptive; matches(raw,hash), one-way.
CORS = browser origin policy (not security). CSRF = forged cookie request; disable for JWT header APIs.
Config(SS6): SecurityFilterChain bean, STATELESS, authorizeHttpRequests, addFilterBefore(jwt,...)
401=unauthenticated, 403=forbidden. @PreAuthorize = method security via AOP.
```

Module 9 — Mock Interview (answer, then continue)

1. "Draw and explain the JWT authentication flow from login to an authorized API call."
2. "Walk through the Spring Security filter chain and where your custom JWT filter goes."
3. "Why do we disable CSRF for a JWT API but not for a session-cookie app?"
4. "Why BCrypt and not SHA-256 for passwords? What does the work factor do?"
5. "How do you handle logout/token revocation in a stateless JWT system?"

Continue to Module 10 when ready.